AJ_24038155_dialect_v1.1_to_v1.2 update

Change specification from AJ_24038155 Dialect v1.1 to v1.2

Status

This document records the v1.1 to v1.2 update to the frozen AJ_24038155 Dialect v1.1 specification. All v1.0 and v1.1 lexical, syntactic and semantic rules remain normative unless explicitly changed below.

Version v1.2 is introduced under the v1.1 versioning rule because it adds canonical nested/local function declarations so that source lexical scope can be preserved without hoisting helper functions to module scope.

Version v1.2 does not add callbacks, first-class function values, anonymous functions, function-reference expressions, or nonlocal-style rebinding. Those capabilities are reserved for a later dialect revision.

1. Nested/local function declarations

1.1 Canonical syntax

A FUNCTION ... ENDFUNCTION declaration may appear inside the body of another function:

```
FUNCTION outer ACCEPTS(value)
   FUNCTION inner ACCEPTS(item)
      RETURN item
   ENDFUNCTION

   RETURN CALL inner(value)
ENDFUNCTION
```

A function declared inside another function is a nested function (also called a local function). The declaration uses the existing canonical function syntax.

1.2 Grammar extension

The existing function declaration remains:

```
function_definition ::=
   "FUNCTION" , identifier , "ACCEPTS" ,
   "(" , [ parameter_list ] , ")" , newline ,
   function_body ,
   "ENDFUNCTION" , newline ;
```

In v1.2 the function body explicitly permits nested function declarations:

```
function_body ::= { function_body_item } ;

function_body_item ::=
   statement
 | function_definition
 | comment ;
```

Because a nested function has an ordinary function body, nesting may be recursive: a nested function may itself declare another nested function.

1.3 Scope and name visibility

A nested function name belongs to the lexical scope in which the declaration appears. It is not promoted to top-level or module scope.

The nested function may call itself recursively by name.

A nested function may read values that are already visible in an enclosing lexical scope. An assignment to a simple identifier inside the nested function creates or updates that nested function's own local binding under the existing assignment rules; v1.2 introduces no nonlocal-style rebinding construct.

Code outside the enclosing function must not rely on the nested function name being visible.

1.4 Calls

Nested functions are invoked with the existing call syntax:

```
CALL inner(argument)
```

No special callback invocation syntax is added.

1.5 Python translation

A nested AJ function translates to a nested Python def at the corresponding lexical indentation.

AJ v1.2:

```
FUNCTION outer ACCEPTS(N)
   FUNCTION helper ACCEPTS(value)
      OUTPUT value
   ENDFUNCTION

   CALL helper(N)
ENDFUNCTION
```

Python:

```
def outer(N):
   def helper(value):
      print(value)

   helper(N)
```

The compiler must not hoist helper to module scope merely to simplify code generation.

2. Callback and first-class function boundary

v1.2 adds declaration nesting only. It does not make function declarations into general expression values.

The following capabilities are not introduced by v1.2:

```
callback <- FUNCTION_REFERENCE(transform)
CALL apply(value, transform)
RETURN transform
APPEND transform TO handlers
```

where transform denotes a function declaration rather than an ordinary data value.

A later dialect revision may define function references, callback parameters, callable-value typing, storage of function values, and any required closure or escape semantics. v1.2 deliberately leaves those questions out of scope.

3. Backwards compatibility

All canonical AJ v1.1 programs remain valid under v1.2.

Top-level FUNCTION declarations retain their existing meaning. Class METHOD and CONSTRUCTOR syntax is unchanged. The v1.2 change only makes lexical nesting of ordinary FUNCTION declarations canonical where the source contains that structure.

Compatibility normalisation should preserve a supplied nested-function relationship when it can be recovered from the source. It must not hoist a nested helper or manufacture a nested helper that the source did not contain.

4. Static semantic additions

1. A nested function belongs to the lexical scope of its enclosing function.

2. A nested function name must not be promoted to top-level/module scope by canonicalisation or compilation.

3. Recursive self-calls from a nested function are permitted.

4. Reads of enclosing visible values are permitted under lexical scope.

5. Assignment to a simple identifier inside the nested function follows the nested function local-binding rule; v1.2 adds no NONLOCAL construct.

6. Function declarations are not first-class values in v1.2, and callback semantics must not be inferred.

5. Tower of Hanoi scope-preservation example

The following structure is canonical AJ v1.2:

```
FUNCTION unnamed_function ACCEPTS(N)
   FUNCTION Hanoi ACCEPTS(Discs, Source, Auxiliary, Destination)
      IF Discs == 1 THEN
         OUTPUT STRING("Move disk from {Source} to {Destination}")
      ELSE
         CALL Hanoi(Discs - 1, Source, Destination, Auxiliary)
         OUTPUT STRING("Move disk from {Source} to {Destination}")
         CALL Hanoi(Discs - 1, Auxiliary, Source, Destination)
      ENDIF
   ENDFUNCTION

   CALL Hanoi(N, STRING("A"), STRING("B"), STRING("C"))
ENDFUNCTION
```

Its deterministic Python structure is:

```
def unnamed_function(N):
   def Hanoi(Discs, Source, Auxiliary, Destination):
      if Discs == 1:
         print(f"Move disk from {Source} to {Destination}")
      else:
         Hanoi(Discs - 1, Source, Destination, Auxiliary)
         print(f"Move disk from {Source} to {Destination}")
         Hanoi(Discs - 1, Auxiliary, Source, Destination)

   Hanoi(N, "A", "B", "C")
```

N remains an externally supplied value. The nested function is not hoisted, and no interactive input prompt is implied by this structure.

6. Normalisation rules

Conservative v1.2 canonicalisation may preserve a source-supplied nested function relationship when lexical nesting is structurally recoverable.

This preservation must be logged and must not invent a helper function, change its lexical scope, manufacture callback semantics, or add nonlocal-style rebinding.

All v1.1 semantic-preservation restrictions remain normative.

7. Version identification

This specification delta is frozen as:

```
Cambridge-Style Pseudocode - AJ_24038155 Dialect v1.2
```

Any later syntax addition, removal or semantic alteration requires a new dialect version.